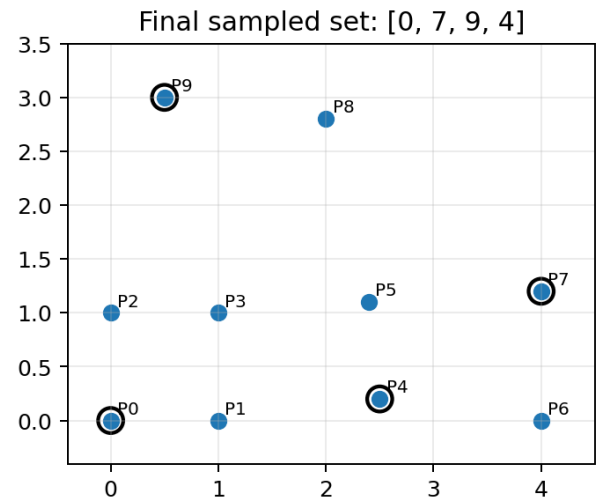
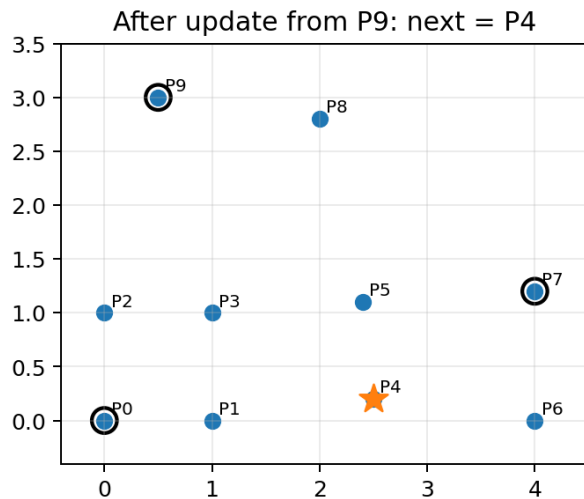
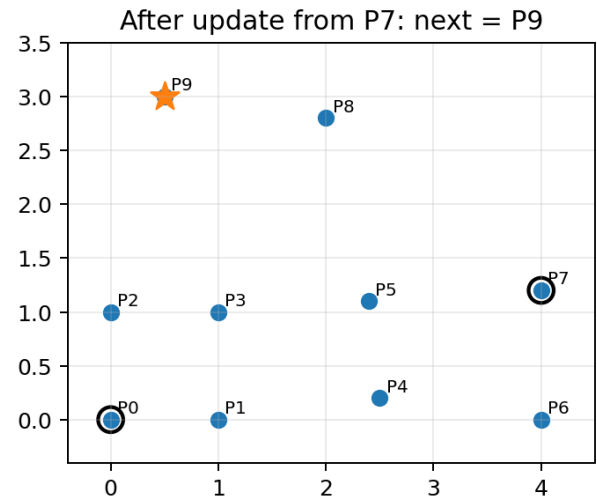
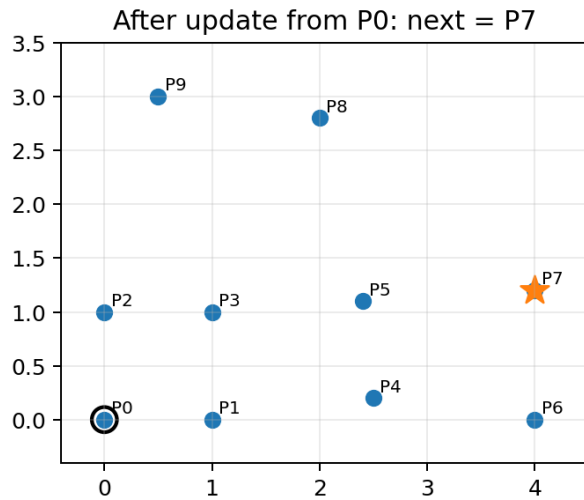


Farthest Point Sampling (FPS) for 3D Point-Cloud Acceleration

Concept, Bottlenecks, RadiusFPS Ideas, and a RISC-V Vector (RVV) Design Path

Toy FPS example (2D for visualization; algorithm is identical in 3D)



Purpose: A technical tutorial for understanding why FPS is difficult to accelerate, what parts vectorize well, what remains serial, and how the ideas in RadiusFPS can guide an RVV-oriented CPU implementation.

Prepared for research and gem5-based architectural exploration

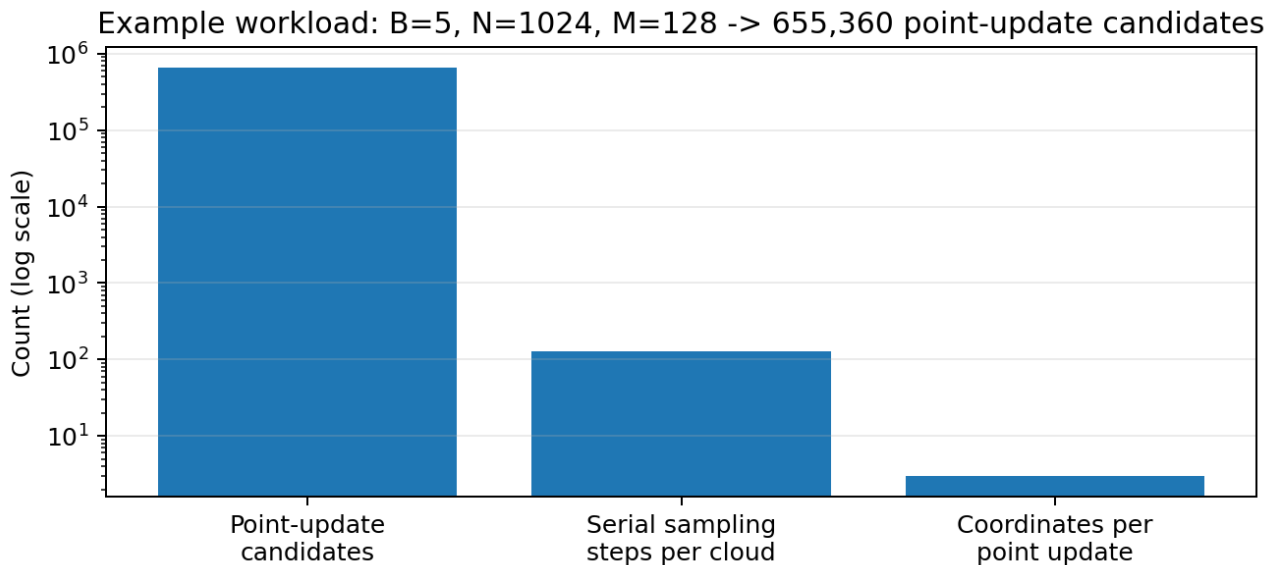
1. Executive summary

Farthest Point Sampling chooses points iteratively so that each new sample is the point whose nearest distance to the already-selected set is maximal. This creates high-quality, spatially well-distributed samples, which is why FPS appears in PointNet++-style pipelines and many later point-cloud models.

The architectural problem is that FPS has two very different kinds of parallelism. **Inside one sampling iteration**, distance updates over N points are highly data-parallel and map naturally to RVV. Across the M sampling iterations, however, there is a strict dependency: iteration $j+1$ cannot know its selected point until iteration j has completed all necessary distance updates and the global argmax.

Main conclusion: RVV can accelerate the inner N -point update, but vanilla RVV still spends vector work on many distances that do not change the state. The RadiusFPS paper is important because it attacks this wasted work before it reaches the arithmetic units, using conservative voxel pruning and point-level skip tests.

- Standard FPS complexity is $O(N*M)$ per point cloud.
- The paper reports that ineffective distance updates exceed 90% in most later FPS iterations on its Stanford Dragon experiment.
- The paper identifies three central problems: high computational overhead, lack of filtering, and inherent sequential dependency.
- For an RVV design, the most promising direction is algorithm-software-hardware co-design: vectorize the remaining useful work, reduce memory traffic, and use pruning/masks to avoid useless work.
- For your PointNet++-style profile $B=5$, $N=1024$, $M=128$, vanilla FPS exposes about 655,360 point-update candidates before considering the global reductions.



2. What FPS is actually computing

Let P contain N points and let S be the sampled set. FPS maintains one state value $D[i]$ per input point. $D[i]$ is the distance from point i to its nearest selected sample. If the newest selected point is s , the update is:

Core update: $D[i] \leftarrow \min(D[i], \text{distance}(P[i], s))$; then choose next = $\text{argmax}_i D[i]$.

The key optimization already present in standard FPS is that we do not recompute the distance to every point in S . We only compare against the most recently added sample and keep the running minimum. This reduces a naive $O(N \cdot M^2)$ interpretation to $O(N \cdot M)$, but $O(N \cdot M)$ is still expensive when N or M is large.

Python reference implementation

```
import numpy as np

def fps(points, m, seed=0):
    """Exact FPS using squared Euclidean distance.
    points: [N, 3] float array
    returns: indices of m sampled points
    """
    n = len(points)
    selected = np.empty(m, dtype=np.int64)
    D = np.full(n, np.inf, dtype=np.float32)

    current = seed
    for j in range(m):
        selected[j] = current

        delta = points - points[current]    # [N, 3]
        d2 = np.sum(delta * delta, axis=1)  # squared L2
        D = np.minimum(D, d2)              # nearest selected sample

        current = int(np.argmax(D))         # strict next-point dependency

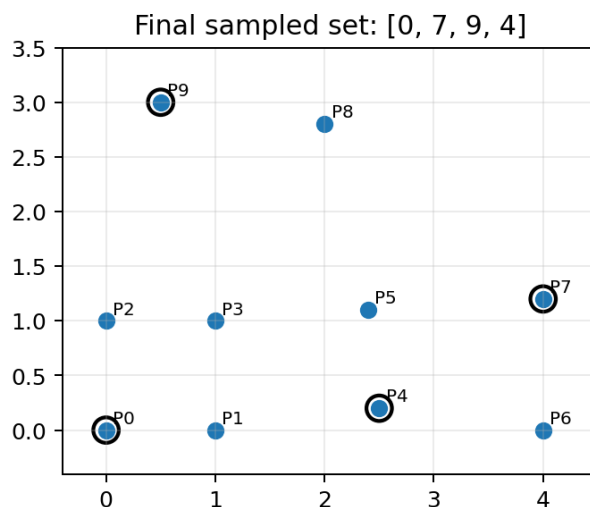
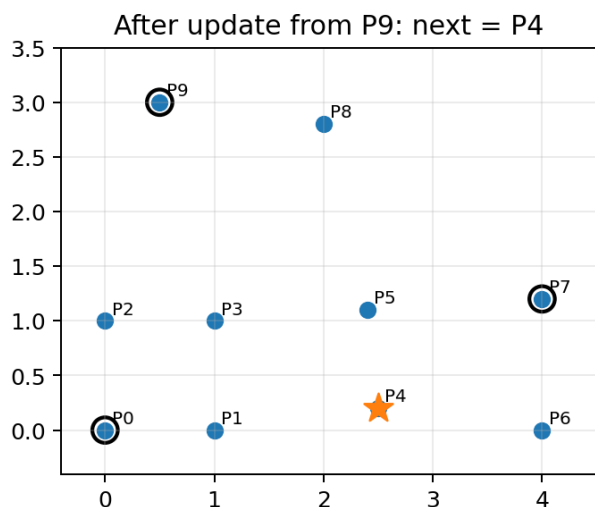
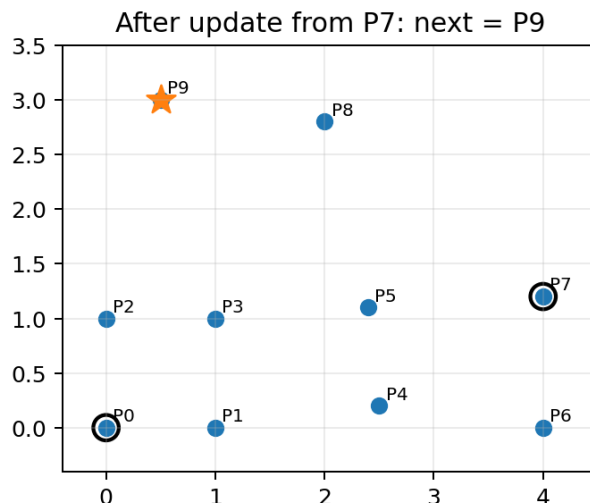
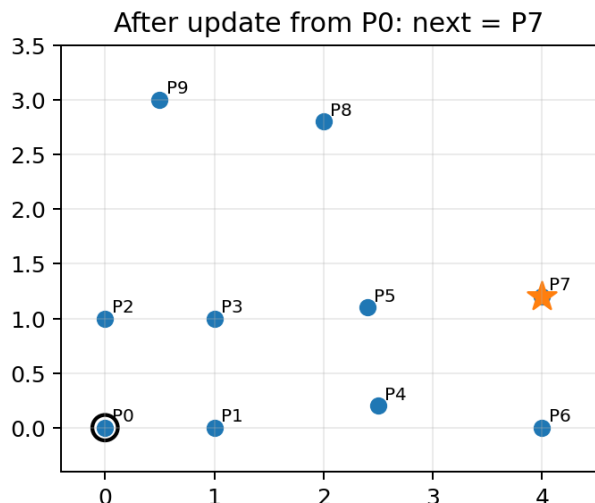
    return selected
```

For FPS selection, squared Euclidean distance is normally sufficient because square root is monotonic: the nearest and farthest ordering is unchanged when all comparisons are done consistently in squared-distance space. This is a useful CPU/RVV optimization because it removes $N \cdot M$ square roots. Exact floating-point tie behavior should still be kept deterministic in benchmarking.

3. Worked example: follow D, not just the selected points

A common source of confusion is thinking that FPS repeatedly finds the point farthest from only the last sample. It does not. The new point is farthest from the entire sampled set, represented compactly by the running array D.

Toy FPS example (2D for visualization; algorithm is identical in 3D)



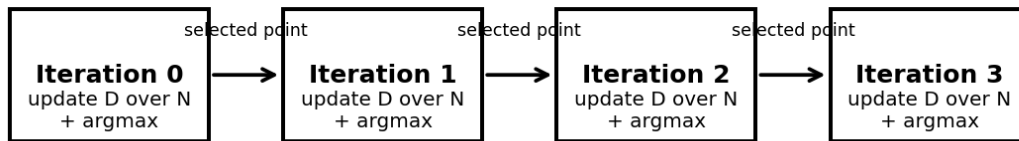
In this toy example the seed is P0 and $M=4$. After each iteration, every $D[i]$ stores the minimum squared distance to any selected point seen so far. The next point is the index of the maximum D value.

Iteration	Newest sample	max(D)	Next sample	D array after update
1	P0	17.44	P7	0.00, 1.00, 1.00, 2.00, 6.29, 6.97, 16.00, 17.44, 11.84, 9.25
2	P7	9.25	P9	0.00, 1.00, 1.00, 2.00, 3.25, 2.57, 1.44, 0.00, 6.56, 9.25
3	P9	3.25	P4	0.00, 1.00, 1.00, 2.00, 3.25, 2.57, 1.44, 0.00, 2.29, 0.00

Why this matters for hardware: The state array D couples all iterations. Any approximation, unsafe pruning decision, or incorrect tie-breaking can change the next sample and therefore change every later iteration.

4. Why FPS is unusually difficult to accelerate

4.1 The outer loop is sequential



The inner N-point update is data-parallel, but the M sampling iterations are a strict dependency chain.

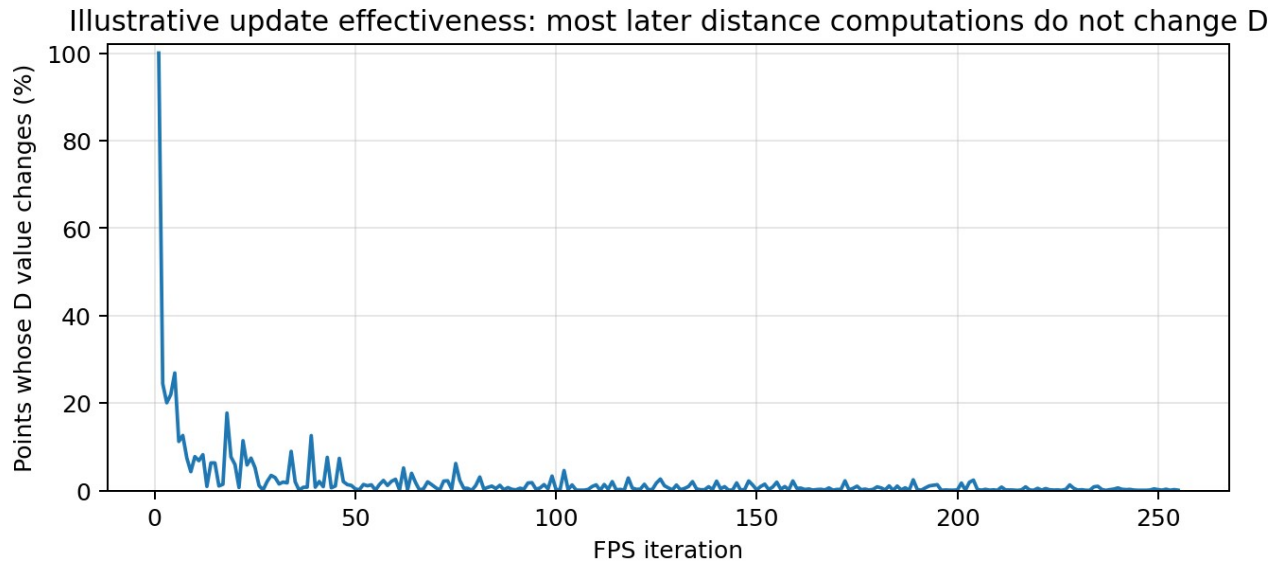
You can parallelize the work over points inside an iteration, but you cannot simply execute multiple future sample-selection iterations simultaneously because the identity of the next sample is unknown until the current D update and argmax finish. This is the fundamental barrier that makes FPS different from embarrassingly parallel pointwise kernels.

4.2 Each iteration touches global state

A straightforward implementation streams point coordinates and the D array, computes a 3D distance, performs a min update, and then performs an argmax reduction. If argmax is implemented as a second pass over D, the same state is read again. For point clouds larger than cache, this becomes a repeated-memory-traffic problem as much as an arithmetic problem.

4.3 Most later updates can be useless

The RadiusFPS paper profiles standard FPS and reports that more than 90% of distance computations become ineffective in later iterations on its Stanford Dragon experiment. An ineffective computation means the new distance is not smaller than the stored nearest-sample distance, so $D[i]$ does not change. The arithmetic and memory access were paid for, but the algorithmic state did not move. In a separate PointMetaBase sampling-pipeline profile, the paper reports that FPS accounts for 77.6% of sampling latency on S3DIS and 89.0% on ScanNet.



Illustrative synthetic example generated for this tutorial; the paper independently reports >90% ineffective updates in most later iterations.

5. Python code to measure the real waste in your own workload

Instrumented FPS

```
import numpy as np

def fps_profile(points, m, seed=0):
    n = len(points)
    D = np.full(n, np.inf, dtype=np.float32)
    current = seed
    selected = []
    stats = []

    for j in range(m):
        selected.append(current)
        delta = points - points[current]
        d2 = np.sum(delta * delta, axis=1)

        changed = d2 < D
        effective = int(np.count_nonzero(changed))
        ineffective = n - effective
        D = np.minimum(D, d2)

        stats.append({
            "iter": j,
            "effective": effective,
            "ineffective": ineffective,
            "effective_pct": 100.0 * effective / n,
            "max_D2": float(D.max()),
        })

        current = int(np.argmax(D))

    return np.array(selected), stats
```

Run this on the same point clouds used in gem5. Plot `effective_pct` versus iteration. This measurement tells you whether your performance problem is mostly useful arithmetic or mostly redundant arithmetic. It is especially important before proposing a new vector instruction: if 90% of lanes perform ineffective work, a faster arithmetic instruction may accelerate the wrong part of the algorithm.

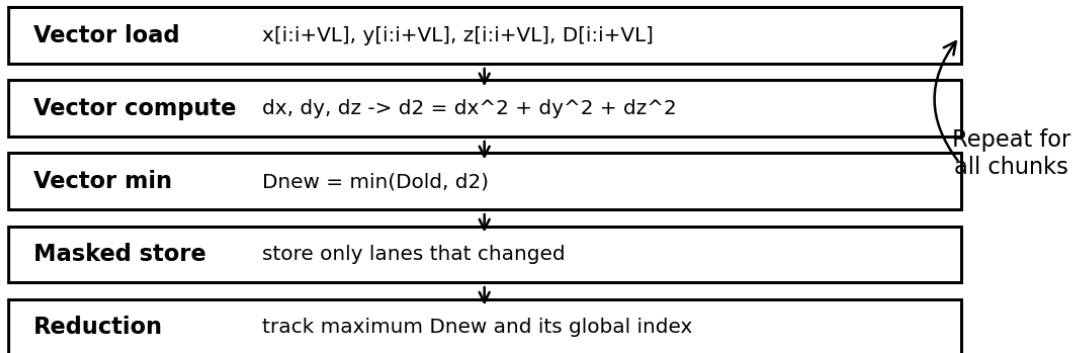
Recommended counter: `effective_update_ratio = (# lanes where  $d2 < D_{old}$ ) / (# point-update candidates).`

Report it alongside cycles and vector instruction counts.

6. Mapping vanilla FPS to RISC-V Vector (RVV)

One serial FPS iteration

broadcast selected point (sx, sy, sz)



RVV accelerates the N-point work inside an iteration; it does not remove the serial M-iteration dependency.

The natural RVV parallel dimension is the point index i . For each selected sample, broadcast its x, y, z coordinates and process the point cloud in VL-sized chunks. The standard V extension provides scalable vector loads/stores, floating arithmetic, masks, and reductions. Segment loads can also unpack array-of-structures data, although a structure-of-arrays layout usually gives the cleanest repeated $x/y/z$ streams.

FPS sub-operation	RVV mapping	Main issue
Load coordinates	vle32 or 3-field segment load	Bandwidth / layout
3D squared distance	vector subtract + multiply/add	Very regular
$D = \min(D, d2)$	vector min	Regular; mask can suppress stores
Find max value	floating max reduction	Reduction serializes chunk result
Recover argmax index	compare-to-max mask + first-set/index tracking	Tie-breaking and index recovery
Next sample	scalar state update	Cannot start next FPS iteration early

A useful software optimization is to fuse the D update and local maximum tracking in the same vector pass. This avoids a separate full scan of D for argmax. In standard RVV, `vfredmax` can reduce the maximum floating-point value. A practical index-recovery sequence is to compare the chunk values against that maximum, form a mask, use `vfirst.m` to obtain the lowest matching lane, and add the chunk base index. Global tie-breaking still has to be handled consistently across chunks.

Important: Compiling with `-march=rv64gcv` only enables RVV. It does not prove that the compiler generated vector instructions. Always inspect the disassembly or use intrinsics for the critical kernel.

7. Why RVV alone may still leave a large bottleneck

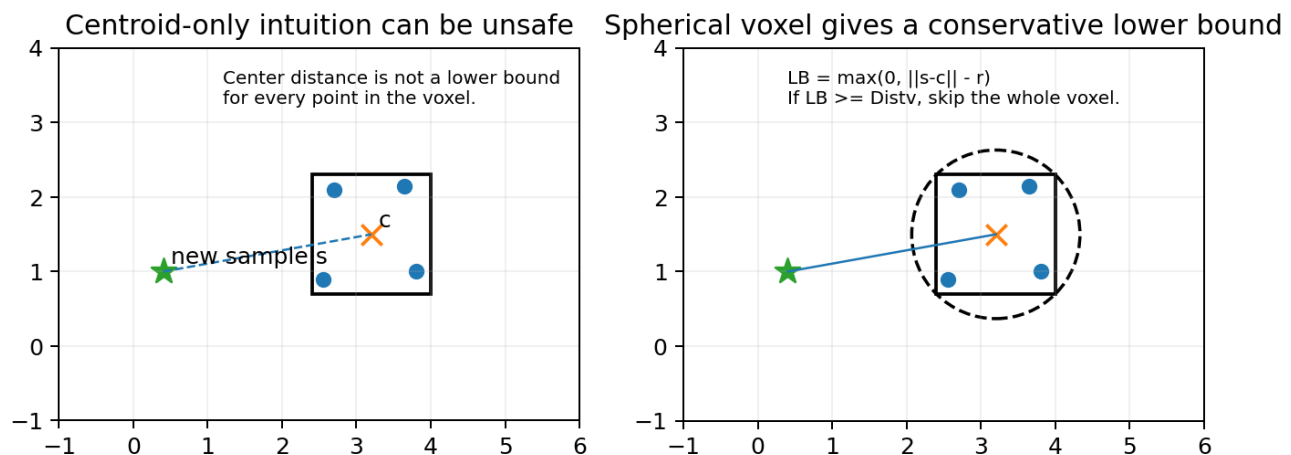
- Serial M-loop remains. Even a perfect vectorized N-point update must complete before the next sample is known.

- Vectorization does not eliminate ineffective distance calculations. It can simply compute the wasted work faster.
- Argmax is not only a value reduction; FPS also needs the index. Exact deterministic tie-breaking adds control logic.
- The D array is repeatedly read and updated. If N is large enough to exceed cache, memory traffic can dominate.
- Masked stores reduce write traffic but do not automatically eliminate the loads and arithmetic that created the mask.
- Irregular pruning can reduce work but may reduce vector occupancy unless surviving points are compacted or organized spatially.

This leads to an important research hypothesis: a good RVV FPS accelerator should not be evaluated only by vector width. It should be evaluated by how much unnecessary work is removed before vector execution and how efficiently the remaining active points are organized.

8. The clever idea in RadiusFPS: prune before computing

The RadiusFPS paper attacks the lack-of-filtering problem with a coarse-to-fine hierarchy. It partitions the cloud into active voxels, maintains point-level nearest distances $Distp$ and voxel-level maxima $Distv$, and uses conservative geometry to decide when a whole voxel cannot possibly contain an effective update.



8.1 Voxel-level radius pruning

For a voxel with center c and bounding-sphere radius r , the paper uses the lower bound $LB = \max(0, ||s-c|| - r)$, where s is the newly selected sample. Because every point in the voxel lies within radius r of c , LB is a conservative lower bound on the distance from s to every point in that voxel.

Safe whole-voxel skip: If $LB \geq Distv$, where $Distv$ is the largest current nearest-sample distance among points in the voxel, then no point in that voxel can receive a smaller D value. The entire voxel can be skipped without changing exact FPS behavior.

8.2 Point-level coordinate skip

For a surviving voxel, the paper applies a cheaper point test before exact Euclidean distance. Since L2 distance is at least the absolute difference along any one coordinate, a point can be skipped if any coordinate difference is already no smaller than its current nearest-sample distance. This avoids squares, adds, square root, and the final distance comparison for many residual points.

8.3 Why the paper is especially relevant to RVV

The paper reports large speedups from reducing redundant work, including up to 186.56x for its CPU RadiusFPS configuration and up to 52.4x for RadiusFPS-G over its GPU FPS baseline in the reported experiments. The exact speedups are workload- and parameter-dependent, but the architectural lesson is strong: search-space reduction can matter more than simply making the distance arithmetic wider.

9. A practical RVV-oriented FPS design path

1. Start with an exact scalar reference using squared distances and deterministic tie-breaking.
2. Implement an RVV inner loop over point index i using SoA $x[], y[], z[], D[]$.
3. Fuse D update and max tracking into one pass to avoid a second full D scan.
4. Add masked stores and measure effective-update ratio; do not assume masks reduce arithmetic.
5. Add the paper-inspired point-skip test. In squared-distance form, a safe version is: skip if $dx^2 \geq D2$ or $dy^2 \geq D2$ or $dz^2 \geq D2$.
6. Add active-voxel organization and conservative voxel pruning. Keep points within an active voxel contiguous so the surviving work remains vector-friendly.
7. Only after the software path is understood, evaluate whether an ISA extension is justified.

Key co-design principle: Pruning reduces the number of lanes that need real distance work; RVV increases throughput on the lanes that remain. The two techniques are complementary, not competing.

Python model for a fused RVV-style iteration

```
# Pedagogical NumPy model of an RVV-like fused update + max pass.
# In C/RVV, each slice corresponds to one variable-length vector chunk.
def fused_fps_step(points, D2, sample_index, vl=32):
    sx, sy, sz = points[sample_index]
    best_value = -1.0
    best_index = -1

    for base in range(0, len(points), vl):
        p = points[base:base+vl]
        old = D2[base:base+vl]

        dx = p[:, 0] - sx
        dy = p[:, 1] - sy
        dz = p[:, 2] - sz
        d2 = dx*dx + dy*dy + dz*dz

        new = np.minimum(old, d2)
        D2[base:base+vl] = new

        lane = int(np.argmax(new))
        value = float(new[lane])
        index = base + lane
        if value > best_value or (value == best_value and index < best_index):
            best_value, best_index = value, index

    return best_index
```

10. Where a custom RISC-V extension might actually help

Before designing a new instruction, compare the best software-only RVV version against scalar and pruning-enhanced variants. The most defensible custom primitives are those that remove instruction overhead or data movement that standard RVV cannot express efficiently.

Candidate idea	Why it helps FPS	Research caution
Argmax-with-index reduction	Returns both maximum D and its lane/index in one reduction	Must define deterministic tie behavior
Fused 3D distance + min update	Collapses repeated subtract/multiply/add/min sequence	Domain-specific; may be too narrow
Fused distance/min + argmax state	Could combine update and selection metadata	Complex semantics and long reduction path
Active-lane compaction support	Improves utilization after pruning	Standard RVV already has masks/compaction primitives; quantify the gap first
PCD-oriented memory primitive	Could streamline x/y/z streams	RVV segment loads already support AoS-style fields, so benefit must be demonstrated

Most promising first hypothesis: A direct max-with-index reduction is more generally useful than an FPS-specific instruction and directly targets a recurring mismatch between reduction value and index recovery.

11. Suggested gem5 experimental plan

Use a staged experiment so you can attribute each speedup to one mechanism rather than comparing a heavily optimized design directly against a weak baseline.

Version	Compile / implementation	Purpose
A. Scalar	rv64gc, O3	Baseline
B. Auto-vector	rv64gcv, O3	Measure compiler auto-vectorization
C. RVV intrinsics	Explicit vector inner loop	Reliable RVV baseline
D. Fused RVV	Update + max tracking in one pass	Reduce D traffic
E. RVV + point skip	Mask cheap coordinate tests	Avoid useless distance math
F. RVV + voxel pruning	RadiusFPS-inspired hierarchy	Avoid whole regions
G. Custom ISA (optional)	Only after E/F are understood	Quantify incremental ISA value

11.1 Metrics worth collecting

- simTicks / cycles and simInsts inside the FPS region of interest.
- Vector instruction count, average VL, and fraction of arithmetic executed under masks.
- L1/L2 data-cache accesses and misses; bytes read/written per selected point.
- Branch count and mispredictions for pruning-heavy variants.
- effective_update_ratio per FPS iteration.
- voxel_prune_ratio and point_skip_ratio for RadiusFPS-inspired versions.
- cycles per point-update candidate and cycles per effective D update.
- Speedup versus scalar for identical N, M, seed, point order, and tie-breaking.

11.2 Workload matrix

Class	Suggested settings	Why
PointNet++-style	N=1024, M=128; N=2048/4096 variants	Matches your current study and exposes scaling
Batch sensitivity	B=1 and B=5	Shows independent-cloud parallelism opportunity
Large PCD	N=50k-250k, several M ratios	Tests cache/memory and paper-like regime
Spatial structure	uniform, clustered, real LiDAR	Pruning effectiveness depends on geometry

12. Concrete research questions for an RVV FPS paper

1. How much speedup does standard RVV provide for exact vanilla FPS when only the N-point inner loop is vectorized?
2. How much of the remaining time is due to argmax/index recovery, memory traffic, or the serial M-loop?
3. Does fusing distance update and maximum tracking materially reduce cache traffic and cycles?
4. How quickly does effective_update_ratio fall for PointNet++-style N=1024 workloads compared with large LiDAR clouds?
5. Can RadiusFPS-style pruning be reorganized to preserve long contiguous RVV vectors rather than producing sparse gathers?
6. At what N and M does voxel-index construction amortize its preprocessing cost?
7. Does a generic argmax-with-index vector primitive provide a meaningful incremental gain after software pruning and loop fusion?
8. How sensitive are speedups to VLEN, LMUL, cache size, memory latency, and point layout (AoS vs SoA)?

Potential paper story: Vanilla FPS is a poor match for "vectorize everything" because the algorithm becomes increasingly dominated by ineffective work and a serial selection chain. A pruning-aware RVV design can convert irregular 3D sampling into fewer, denser vector updates, while a small reduction-oriented ISA enhancement can address the remaining selection overhead.

13. Source notes and references

Paper-derived statements in this tutorial are based on the uploaded RadiusFPS preprint. Architectural mapping to RVV, squared-distance reformulation, loop fusion, gem5 methodology, and candidate ISA extensions are analysis/recommendations developed for your stated RISC-V vector research direction, not claims made by the RadiusFPS paper.

- [1] Ziyang Yu, Xiang Li, Qiong Chang, Jun Miyazaki. "RadiusFPS: Efficient Farthest Point Sampling on CPUs and GPUs via Spherical Voxel Pruning." arXiv:2606.06255, 2026.
- [2] RISC-V International. "V Standard Extension for Vector Operations, Version 1.0." RISC-V Ratified Specifications Library.
- [3] RISC-V International. "RISC-V Vector C Intrinsic Specification." RISC-V Ratified Specifications Library.